2.12 stdio.h

The stdio header provides functions for performing input and output.

Macros:

```
NULL
_IOFBF
_IOLBF
_IONBF
BUFSIZ
EOF
FOPEN_MAX
FILENAME_MAX
L_tmpnam
SEEK_CUR
SEEK_END
SEEK_SET
TMP_MAX
stderr
stdin
stdout
```

Functions:

```
clearerr();
fclose();
feof();
ferror();
fflush();
fgetpos();
fopen();
fread();
freopen();
fseek();
fsetpos();
ftell();
fwrite();
remove();
rename();
rewind();
setbuf();
setvbuf();
tmpfile();
tmpnam();
fprintf();
fscanf();
printf();
scanf();
sprintf();
sscanf();
vfprintf();
vprintf();
vsprintf();
fgetc();
fgets();
fputc();
```

```
fputs();
getc();
getchar();
gets();
putc();
putchar();
puts();
ungetc();
perror();
```

Variables:

```
typedef size_t
typedef FILE
typedef fpos_t
```

2.12.1 Variables and Definitions

`size_t` is the unsigned integer result of the sizeof keyword.
`FILE` is a type suitable for storing information for a file stream.
`fpos_t` is a type suitable for storing any position in a file.

`NULL` is the value of a null pointer constant.
`_IOFBF`, `_IOLBF`, and `_IONBF` are used in the setvbuf function.
`BUFSIZ` is an integer which represents the size of the buffer used by the setbuf function.
`EOF` is a negative integer which indicates an end-of-file has been reached.
`FOPEN_MAX` is an integer which represents the maximum number of files that the system can guarantee that can be opened simultaneously.
`FILENAME_MAX` is an integer which represents the longest length of a char array suitable for holding the longest possible filename. If the implementation imposes no limit, then this value should be the recommended maximum value.
`L_tmpnam` is an integer which represents the longest length of a char array suitable for holding the longest possible temporary filename created by the tmpnam function.
`SEEK_CUR`, `SEEK_END`, and `SEEK_SET` are used in the fseek function.
`TMP_MAX` is the maximum number of unique filenames that the function tmpnam can generate.
`stderr`, `stdin`, and `stdout` are pointers to `FILE` types which correspond to the standard error, standard input, and standard output streams.

2.12.2 Streams and Files

Streams facilitate a way to create a level of abstraction between the program and an input/output device. This allows a common method of sending and receiving data amongst the various types of devices available. There are two types of streams: text and binary.

Text streams are composed of lines. Each line has zero or more characters and are terminated by a new-line character which is the last character in a line. Conversions may occur on text streams during input and output. Text streams consist of only printable characters, the tab character, and the new-line character. Spaces cannot appear before a newline character, although it is implementation-defined whether or not reading a text stream removes these spaces. An implementation must support lines of up to at least 254 characters including the new-line character.

Binary streams input and output data in an exactly 1:1 ratio. No conversion exists and all characters may be transferred.

When a program begins, there are already three available streams: standard input, standard output, and standard error.

Files are associated with streams and must be opened to be used. The point of I/O within a file is determined by the file position. When a file is opened, the file position points to the beginning of the file unless the file is opened for an append operation in which case the position points to the end of the file. The file position follows read and write operations to indicate where the next operation will occur.

When a file is closed, no more actions can be taken on it until it is opened again. Exiting from the main function causes all open files to be closed.

2.12.3 File Functions

2.12.3.1 clearerr

Declaration:

```
void clearerr(FILE *stream);
```

Clears the end-of-file and error indicators for the given stream. As long as the error indicator is set, all stream operations will return an error until **clearerr** or `rewind` is called.

2.12.3.2 fclose

Declaration:

```
int fclose(FILE *stream);
```

Closes the stream. All buffers are flushed.

If successful, it returns zero. On error it returns `EOF`.

2.12.3.3 feof

Declaration:

```
int feof(FILE *stream);
```

Tests the end-of-file indicator for the given stream. If the stream is at the end-of-file, then it returns a nonzero value. If it is not at the end of the file, then it returns zero.

2.12.3.4 ferror

Declaration:

```
int ferror(FILE *stream);
```

Tests the error indicator for the given stream. If the error indicator is set, then it returns a nonzero value. If the error indicator is not set, then it returns zero.

2.12.3.5 fflush

Declaration:

```
int fflush(FILE *stream);
```

Flushes the output buffer of a stream. If stream is a null pointer, then all output buffers are flushed.

If successful, it returns zero. On error it returns `EOF`.

2.12.3.6 fgetpos

Declaration:

```
int fgetpos(FILE *stream, fpos_t *pos);
```

Gets the current file position of the stream and writes it to *pos*.

If successful, it returns zero. On error it returns a nonzero value and stores the error number in the variable `errno`.

2.12.3.7 fopen

Declaration:

```
FILE *fopen(const char *filename, const char *mode);
```

Opens the filename pointed to by filename. The mode argument may be one of the following constant strings:

`r`	read text mode
`w`	write text mode (truncates file to zero length or creates new file)
`a`	append text mode for writing (opens or creates file and sets file pointer to the end-of-file)
`rb`	read binary mode
`wb`	write binary mode (truncates file to zero length or creates new file)
`ab`	append binary mode for writing (opens or creates file and sets file pointer to the end-of-file)
`r+`	read and write text mode
`w+`	read and write text mode (truncates file to zero length or creates new file)
`a+`	read and write text mode (opens or creates file and sets file pointer to the end-of-file)
`r+b` or `rb+`	read and write binary mode
`w+b` or `wb+`	read and write binary mode (truncates file to zero length or creates new file)

`a+b` or `ab+` read and write binary mode (opens or creates file and sets file pointer to the end-of-file)

If the file does not exist and it is opened with read mode (`r`), then the open fails.

If the file is opened with append mode (`a`), then all write operations occur at the end of the file regardless of the current file position.

If the file is opened in the update mode (`+`), then output cannot be directly followed by input and input cannot be directly followed by output without an intervening fseek, fsetpos, rewind, or fflush.

On success a pointer to the file stream is returned. On failure a null pointer is returned.

2.12.3.8 fread

Declaration:

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Reads data from the given stream into the array pointed to by *ptr*. It reads *nmemb* number of elements of size *size*. The total number of bytes read is (`size*nmemb`).

On success the number of elements read is returned. On error or end-of-file the total number of elements successfully read (which may be zero) is returned.

2.12.3.9 freopen

Declaration:

```
FILE *freopen(const char *filename, const char *mode, FILE *stream);
```

Associates a new filename with the given open stream. The old file in stream is closed. If an error occurs while closing the file, the error is ignored. The mode argument is the same as described in the fopen command. Normally used for reassociating stdin, stdout, or stderr.

On success the pointer to the stream is returned. On error a null pointer is returned.

2.12.3.10 fseek

Declaration:

```
int fseek(FILE *stream, long int offset, int whence);
```

Sets the file position of the stream to the given offset. The argument *offset* signifies the number of bytes to seek from the given whence position. The argument *whence* can be:

`SEEK_SET` Seeks from the beginning of the file.

`SEEK_CUR` Seeks from the current position.

`SEEK_END` Seeks from the end of the file.

On a text stream, whence should be `SEEK_SET` and *offset* should be either zero or a value returned from `ftell`.

The end-of-file indicator is reset. The error indicator is NOT reset.

On success zero is returned. On error a nonzero value is returned.

2.12.3.11 fsetpos

Declaration:

```
int fsetpos(FILE *stream, const fpos_t *pos);
```

Sets the file position of the given stream to the given position. The argument *pos* is a position given by the function `fgetpos`. The end-of-file indicator is cleared.

On success zero is returned. On error a nonzero value is returned and the variable `errno` is set.

2.12.3.12 ftell

Declaration:

```
long int ftell(FILE *stream);
```

Returns the current file position of the given stream. If it is a binary stream, then the value is the number of bytes from the beginning of the file. If it is a text stream, then the value is a value useable by the fseek function to return the file position to the current position.

On success the current file position is returned. On error a value of `-1L` is returned and `errno` is set.

2.12.3.13 fwrite

Declaration:

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Writes data from the array pointed to by *ptr* to the given stream. It writes *nmemb* number of elements of size *size*. The total number of bytes written is (`size*nmemb`).

On success the number of elements writen is returned. On error the total number of elements successfully writen (which may be zero) is returned.

2.12.3.14 remove

Declaration:

```
int remove(const char *filename);
```

Deletes the given filename so that it is no longer accessible (unlinks the file). If the file is currently open, then the result is implementation-defined.

On success zero is returned. On failure a nonzero value is returned.

2.12.3.15 rename

Declaration:

```
int rename(const char *old_filename, const char *new_filename);
```

Causes the filename referred to by *old_filename* to be changed to *new_filename*. If the filename pointed to by *new_filename* exists, the result is implementation-defined.

On success zero is returned. On error a nonzero value is returned and the file is still accessible by its old filename.

2.12.3.16 rewind

Declaration:

```
void rewind(FILE *stream);
```

Sets the file position to the beginning of the file of the given stream. The error and end-of-file indicators are reset.

2.12.3.17 setbuf

Declaration:

```
void setbuf(FILE *stream, char *buffer);
```

Defines how a stream should be buffered. This should be called after the stream has been opened but before any operation has been done on the stream. Input and output is fully buffered. The default `BUFSIZ` is the size of the buffer. The argument *buffer* points to an array to be used as the buffer. If *buffer* is a null pointer, then the stream is unbuffered.

2.12.3.18 setvbuf

Declaration:

```
int setvbuf(FILE *stream, char *buffer, int mode, size_t size);
```

Defines how a stream should be buffered. This should be called after the stream has been opened but before any operation has been done on the stream. The argument *mode* defines how the stream should be buffered as follows:

`_IOFBF`
: Input and output is fully buffered. If the buffer is empty, an input operation attempts to fill the

buffer. On output the buffer will be completely filled before any information is written to the file (or the stream is closed).

`_IOLBF` Input and output is line buffered. If the buffer is empty, an input operation attempts to fill the buffer. On output the buffer will be flushed whenever a newline character is written.

`_IONBF` Input and output is not buffered. No buffering is performed.

The argument *buffer* points to an array to be used as the buffer. If *buffer* is a null pointer, then `setvbuf` uses `malloc` to create its own buffer.

The argument *size* determines the size of the array.

On success zero is returned. On error a nonzero value is returned.

2.12.3.19 tmpfile

Declaration:

```
FILE *tmpfile(void);
```

Creates a temporary file in binary update mode (wb+). The tempfile is removed when the program terminates or the stream is closed.

On success a pointer to a file stream is returned. On error a null pointer is returned.

2.12.3.20 tmpnam

Declaration:

```
char *tmpnam(char *str);
```

Generates and returns a valid temporary filename which does not exist. Up to `TMP_MAX` different filenames can be generated.

If the argument *str* is a null pointer, then the function returns a pointer to a valid filename. If the argument *str* is a valid pointer to an array, then the filename is written to the array and a pointer to the same array is returned. The filename may be up to `L_tmpnam` characters long.

2.12.4 Formatted I/O Functions

2.12.4.1 ..printf Functions

Declarations:

```
int fprintf(FILE *stream, const char *format, ...);
int printf(const char *format, ...);
int sprintf(char *str, const char *format, ...);
int vfprintf(FILE *stream, const char *format, va_list arg);
```

```
int vprintf(const char *format, va_list arg);
int vsprintf(char *str, const char *format, va_list arg);
```

The ..printf functions provide a means to output formatted information to a stream.

`fprintf`	sends formatted output to a stream
`printf`	sends formatted output to stdout
`sprintf`	sends formatted output to a string
`vfprintf`	sends formatted output to a stream using an argument list
`vprintf`	sends formatted output to stdout using an argument list
`vsprintf`	sends formatted output to a string using an argument list

These functions take the format string specified by the *format* argument and apply each following argument to the format specifiers in the string in a left to right fashion. Each character in the format string is copied to the stream except for conversion characters which specify a format specifier.

The string commands (`sprintf` and `vsprintf`) append a null character to the end of the string. This null character is not counted in the character count.

The argument list commands (`vfprintf`, `vprintf`, and `vsprintf`) use an argument list which is prepared by `va_start`. These commands do not call `va_end` (the caller must call it).

A conversion specifier begins with the `%` character. After the `%` character come the following in this order:

[**flags**]	Control the conversion (optional).
[**width**]	Defines the number of characters to print (optional).
[**.precision**]	Defines the amount of precision to print for a number type (optional).
[**modifier**]	Overrides the size (type) of the argument (optional).
[**type**]	The type of conversion to be applied (required).

Flags:

`-`	Value is left justified (default is right justified). Overrides the 0 flag.
`+`	Forces the sign (+ or -) to always be shown. Default is to just show the - sign. Overrides the space flag.
`space`	Causes a positive value to display a space for the sign. Negative values still show the - sign.
`#`	Alternate form:

Conversion Character	Result
`o`	Precision is increased to make the first digit a zero.
`X or x`	Nonzero value will have 0x or 0X prefixed to it.
`E, e, f, g, or G`	Result will always have a decimal point.
`G or g`	Trailing zeros will not be removed.

`0`	For d, i, o, u, x, X, e, E, f, g, and G leading zeros are used to pad the field width instead of spaces. This is useful only with a width specifier. Precision overrides this flag.

Width:

The width of the field is specified here with a decimal value. If the value is not large enough to fill the width, then the rest of the field is padded with spaces (unless the 0 flag is specified). If the value overflows the width of the field, then the field is expanded to fit the value. If a `*` is used in place of the width specifer, then the next argument (which must be an `int` type) specifies the width of the field. Note: when using the `*` with the width and/or precision specifier, the width argument comes first, then the precision argument, then the value to be converted.

Precision:
The precision begins with a dot (.) to distinguish itself from the width specifier. The precision can be given as a decimal value or as an asterisk (`*`). If a `*` is used, then the next argument (which is an `int` type) specifies the precision. Note: when using the `*` with the width and/or precision specifier, the width argument comes first, then the precision argument, then the value to be converted. Precision does not affect the c type.

[.precision]	**Result**
(*none*)	Default precision values: 1 for `d`, `i`, `o`, `u`, `x`, `X` types. The minimum number of digits to appear. 6 for `f`, `e`, `E` types. Specifies the number of digits after the decimal point. For `g` or `G` types all significant digits are shown. For `s` type all characters in string are print up to but not including the null character.
`.` *or* `.0`	For `d`, `i`, `o`, `u`, `x`, `X` types the default precis ion value is used unless the value is zero in which case no characters are printed. For `f`, `e`, `E` types no decimal point character or digits are printed. For `g` or `G` types the precision is assumed to be 1.
`.`*n*	For `d`, `i`, `o`, `u`, `x`, `X` types then at least n digits are printed (padding with zeros if necessary). For `f`, `e`, `E` types specifies the number of digits after the decimal point. For `g` or `G` types specifies the number of significant digits to print. For `s` type specifies the maximum number of characters to print.

Modifier:
A modifier changes the way a conversion specifier type is interpreted.

[modifier]	**[type]**	**Effect**
`h`	`d`, `i`, `o`, `u`, `x`, `X`	Value is first converted to a short int or unsigned short i nt.
`h`	`n`	Specifies that the pointer points to a short int.
`l`	`d`, `i`, `o`, `u`, `x`, `X`	Value is first converted to a long int or unsigned long int .
`l`	`n`	Specifies that the pointer points to a long int.
`L`	`e`, `E`, `f`, `g`, `G`	Value is first converted to a long double.

Conversion specifier type:
The conversion specifier specifies what type the argument is to be treated as.

[type]	**Output**
`d`, `i`	Type `signed int`.
`o`	Type `unsigned int` printed in octal.
`u`	Type `unsigned int` printed in decimal.
`x`	Type `unsigned int` printed in hexadecimal as dddd using a, b, c, d, e, f.
`X`	Type `unsigned int` printed in hexadecimal as dddd using A, B, C, D, E, F.
`f`	Type `double` printed as [-]ddd.ddd.

`e, E` Type `double` printed as [-]d.ddde□where there is one digit printed before the decimal (zero only if the value is zero). The exponent contains at least two digits. If type is E then the exponent is printed with a capital E.

`g, G` Type `double` printed as type e or E if the exponent is less than -4 or greater than or equal to the precision. Otherwise printed as type f. Trailing zeros are removed. Decimal point character appears only if there is a nonzero decimal digit.

`c` Type `char`. Single character is printed.

`s` Type pointer to array. String is printed according to precision (no precision prints entire string).

`p` Prints the value of a pointer (the memory location it holds).

`n` The argument must be a pointer to an `int`. Stores the number of characters printed thus far in the int. No characters are printed.

`%` A `%` sign is printed.

The number of characters printed are returned. If an error occurred, -1 is returned.

2.12.4.2 ..scanf Functions

Declarations:

```
int fscanf(FILE *stream, const char *format, ...);
int scanf(const char *format, ...);
int sscanf(const char *str, const char *format, ...);
```

The ..scanf functions provide a means to input formatted information from a stream.

`fscanf` reads formatted input from a stream

`scanf` reads formatted input from stdin

`sscanf` reads formatted input from a string

These functions take input in a manner that is specified by the format argument and store each input field into the following arguments in a left to right fashion.

Each input field is specified in the format string with a conversion specifier which specifies how the input is to be stored in the appropriate variable. Other characters in the format string specify characters that must be matched from the input, but are not stored in any of the following arguments. If the input does not match then the function stops scanning and returns. A whitespace character may match with any whitespace character (space, tab, carriage return, new line, vertical tab, or formfeed) or the next incompatible character.

An input field is specified with a conversion specifer which begins with the `%` character. After the `%` character come the following in this order:

[*] Assignment suppressor (optional).

[width] Defines the maximum number of characters to read (optional).

[modifier] Overrides the size (type) of the argument (optional).

[type] The type of conversion to be applied (required).

Assignment suppressor:

Causes the input field to be scanned but not stored in a variable.

Width:
The maximum width of the field is specified here with a decimal value. If the input is smaller than the width specifier (i.e. it reaches a nonconvertible character), then what was read thus far is converted and stored in the variable.

Modifier:
A modifier changes the way a conversion specifier type is interpreted.

[modifier]	[type]	Effect
`h`	`d`, `i`, `o`, `u`, `x`	The argument is a `short int` or `unsigned short int`.< /td>
`h`	`n`	Specifies that the pointer points to a `short int`.
`l`	`d`, `i`, `o`, `u`, `x`	The argument is a `long int` or `unsigned long int` .
`l`	`n`	Specifies that the pointer points to a `long int`.
`l`	`e`, `f`, `g`	The argument is a `double`.
`L`	`e`, `f`, `g`	The argument is a `long double`.

Conversion specifier type:
The conversion specifier specifies what type the argument is. It also controls what a valid convertible character is (what kind of characters it can read so it can convert to something compatible).

[type]	Input
`d`	Type `signed int` represented in base 10. Digits 0 through 9 and the sign (+ or -).
`i`	Type `signed int`. The base (radix) is dependent on the first two characters. If the first character is a digit from 1 to 9, then it is base 10. If the first digit is a zero and the second digit is a digit from 1 to 7, then it is base 8 (octal). If the first digit is a zero and the second character is an x or X, then it is base 16 (hexadecimal).
`o`	Type `unsigned int`. The input must be in base 8 (octal). Digits 0 through 7 only.
`u`	Type `unsigned int`. The input must be in base 10 (decimal). Digits 0 through 9 only.
`x, X`	Type `unsigned int`. The input must be in base 16 (hexadecimal). Digits 0 through 9 or A through Z or a through z. The characters 0x or 0X may be optionally prefixed to the value.
`e`, `E`, `f`, `g`, `G`	Type `float`. Begins with an optional sign. Then one or more digits, followed by an optional decimal-point and decimal value. Finally ended with an optional signed exponent value designated with an e or E.
`s`	Type character array. Inputs a sequence of non-whitespace characters (space, tab, carriage return, new line, vertical tab, or formfeed). The array must be large enough to hold the sequence plus a null character appended to the end.
`[...]`	Type character array. Allows a search set of characters. Allows input of only those character encapsulated in the brackets (the scanset). If the first character is a carrot (^), then the scanset is inverted and allows any ASCII character except those specified between the brackets. On some systems a range can be specified with the dash character (-). By specifying the beginning character, a dash, and an ending character a range of characters can be included in the scanset. A null character is appended to the end of the array.
`c`	Type character array. Inputs the number of characters specified in the width field. If no width field is specified, then 1 is assumed. No null character is appended to the array.
`p`	Pointer to a pointer. Inputs a memory address in the same fashion of the `%p` type produced by the printf function.

`n`	The argument must be a pointer to an `int`. Stores the number of characters read thus far in the `int`. No characters are read from the input stream.
`%`	Requires a matching `%` sign from the input.

Reading an input field (designated with a conversion specifier) ends when an incompatible character is met, or the width field is satisfied.

On success the number of input fields converted and stored are returned. If an input failure occurred, then EOF is returned.

2.12.5 Character I/O Functions

2.12.5.1 fgetc

Declaration:

```
int fgetc(FILE *stream);
```

Gets the next character (an `unsigned char`) from the specified stream and advances the position indicator for the stream.

On success the character is returned. If the end-of-file is encountered, then `EOF` is returned and the end-of-file indicator is set. If an error occurs then the error indicator for the stream is set and `EOF` is returned.

2.12.5.2 fgets

Declaration:

```
char *fgets(char *str, int n, FILE *stream);
```

Reads a line from the specified stream and stores it into the string pointed to by *str*. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first. The newline character is copied to the string. A null character is appended to the end of the string.

On success a pointer to the string is returned. On error a null pointer is returned. If the end-of-file occurs before any characters have been read, the string remains unchanged.

2.12.5.3 fputc

Declaration:

```
int fputc(int char, FILE *stream);
```

Writes a character (an `unsigned char`) specified by the argument *char* to the specified stream and advances the position indicator for the stream.

On success the character is returned. If an error occurs, the error indicator for the stream is set and `EOF` is

returned.

2.12.5.4 fputs

Declaration:

```
int fputs(const char *str, FILE *stream);
```

Writes a string to the specified stream up to but not including the null character.

On success a nonnegative value is returned. On error `EOF` is returned.

2.12.5.5 getc

Declaration:

```
int getc(FILE *stream);
```

Gets the next character (an `unsigned char`) from the specified stream and advances the position indicator for the stream.

This may be a macro version of `fgetc`.

On success the character is returned. If the end-of-file is encountered, then `EOF` is returned and the end-of-file indicator is set. If an error occurs then the error indicator for the stream is set and `EOF` is returned.

2.12.5.6 getchar

Declaration:

```
int getchar(void);
```

Gets a character (an `unsigned char`) from `stdin`.

On success the character is returned. If the end-of-file is encountered, then `EOF` is returned and the end-of-file indicator is set. If an error occurs then the error indicator for the stream is set and `EOF` is returned.

2.12.5.7 gets

Declaration:

```
char *gets(char *str);
```

Reads a line from `stdin` and stores it into the string pointed to by *str*. It stops when either the newline character is read or when the end-of-file is reached, whichever comes first. The newline character is not copied to the string. A null character is appended to the end of the string.

On success a pointer to the string is returned. On error a null pointer is returned. If the end-of-file occurs before any characters have been read, the string remains unchanged.

2.12.5.8 putc

Declaration:

```
int putc(int char, FILE *stream);
```

Writes a character (an `unsigned char`) specified by the argument *char* to the specified stream and advances the position indicator for the stream.

This may be a macro version of `fputc`.

On success the character is returned. If an error occurs, the error indicator for the stream is set and `EOF` is returned.

2.12.5.9 putchar

Declaration:

```
int putchar(int char);
```

Writes a character (an `unsigned char`) specified by the argument *char* to `stdout`.

On success the character is returned. If an error occurs, the error indicator for the stream is set and `EOF` is returned.

2.12.5.10 puts

Declaration:

```
int puts(const char *str);
```

Writes a string to `stdout` up to but not including the null character. A newline character is appended to the output.

On success a nonnegative value is returned. On error `EOF` is returned.

2.12.5.11 ungetc

Declaration:

```
int ungetc(int char, FILE *stream);
```

Pushes the character *char* (an `unsigned char`) onto the specified stream so that the this is the next character read. The functions `fseek`, `fsetpos`, and `rewind` discard any characters pushed onto the

stream.

Multiple characters pushed onto the stream are read in a FIFO manner (first in, first out).

On success the character pushed is returned. On error `EOF` is returned.

2.12.7 Error Functions

2.12.7.1 perror

Declaration:

```
void perror(const char *str);
```

Prints a descriptive error message to stderr. First the string *str* is printed followed by a colon then a space. Then an error message based on the current setting of the variable `errno` is printed.